

MusicGraph

Exploration des collaborations musicales

Projet NoSQL — B3 Dev & Data

Adlen Souci, Jade Hurpet, Sophie Aholou

Sommaire

01 Contexte & Problématique

02 Architecture du projet

03 Docker & Déploiement

04 Modèle de données Neo4j

05 API Backend (NestJS)

06 Intégration MusicBrainz

07 Interface web (React)

08 Analyse des données

09 Qualité des données

10 Conclusion

Contexte & Problématique

Le problème

Les plateformes musicales contiennent des relations complexes : artistes, morceaux, albums, featurings, collaborations. Difficile à représenter en base relationnelle (SQL).

La solution

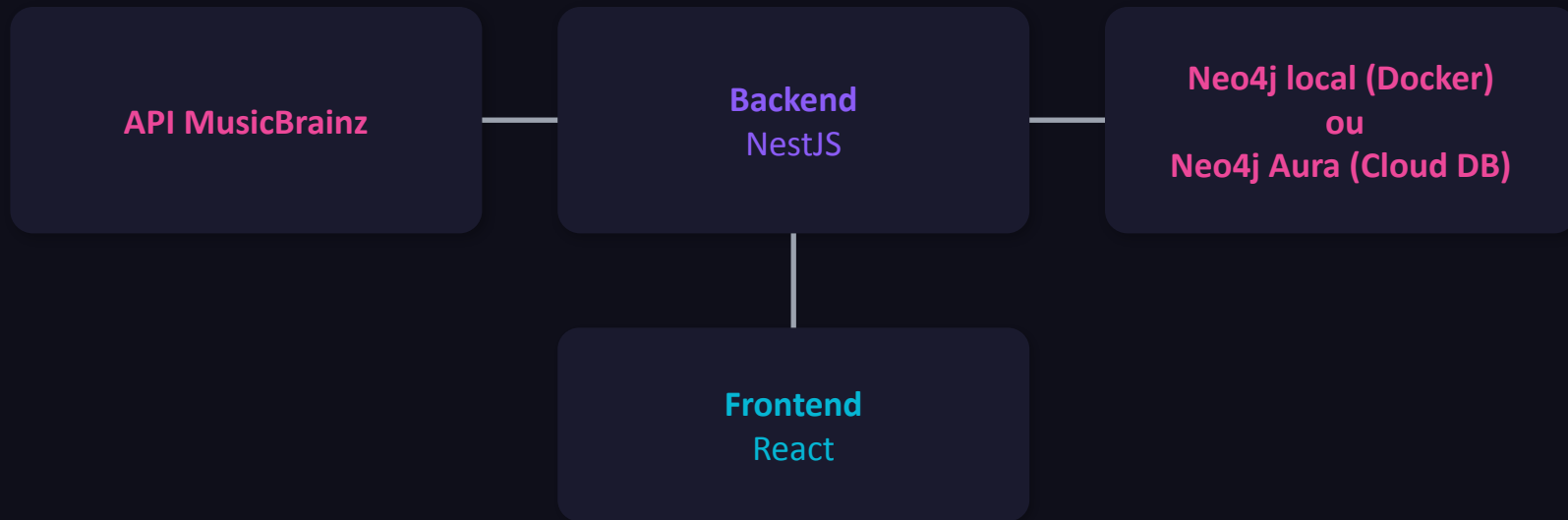
Base de données orientée graphe (Neo4j) : les relations entre entités sont natives, pas besoin de jointures complexes.

Problématique

Comment construire un graphe permettant d'explorer :

- les morceaux associés à un artiste ?
- les collaborations et featurings ?
- les albums contenant ces morceaux ?
- les artistes les plus connectés ?
- les genres les plus présents ?
- les chemins reliant deux artistes ?

Architecture du projet



[backend/](#) NestJS API [frontend/](#) React SPA [data/](#) Seeds & exports [docs/](#) Documentation

Docker & Déploiement

`docker-compose.yml`

3 services orchestrés : backend + frontend + Neo4J

Backend

Node 20 Alpine → npm build → npm start

Frontend

Build React (Vite) → serveur Nginx

Nginx

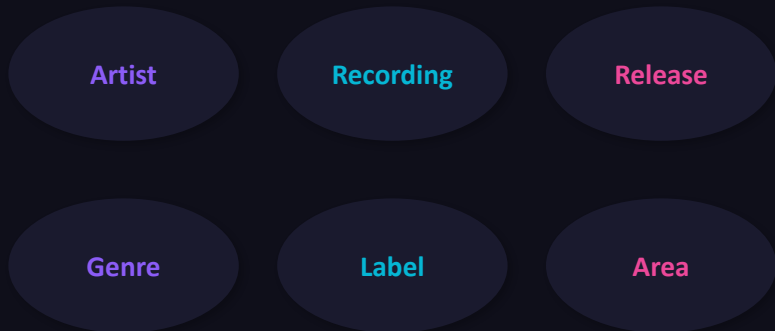
SPA routing + proxy API vers le backend

`.env.example`

Template de configuration fourni

Modèle de données Neo4j

6 types de noeuds



8 relations

PERFORMED
FEATURED_ON
COLLABORATED_WITH
APPEARS_ON
RELEASED_BY
ASSOCIATED_WITH_GENRE
FROM_AREA
RELEASED_IN

Exemple

```
(Booba) - [:PERFORMED] -> (DKR)  
(Drake) - [:COLLABORATED_WITH] -> (21 Savage)
```

API Backend (NestJS)

20+

endpoints REST

Modules

Artists

Recordings

Releases

Graph

Stats

MusicBrainz

Endpoints principaux

```
GET /api/artists
POST /api/import/artists
GET /api/artists/:id
GET /api/search/artists
```

```
GET /api/graph
GET /api/stats/overview
GET /api/recordings
GET /api/releases
```

Intégration MusicBrainz

API publique

Des millions d'artistes, morceaux et albums disponibles gratuitement

Rate limiting

1.1 seconde entre chaque requête pour respecter les limites

Données récupérées

Artiste (nom, pays, genres), recordings, releases, artist-credit

Détection featurings

Analyse des crédits et titres (feat., ft., avec, &, x)

Interface web (React)

7

pages

Accueil

Recherche

Artistes

Détail artiste

Morceaux

Graphe

Statistiques

Dark theme immersif

Particules animées, glow effects, equalizer bars

Filtres dynamiques

Par nom, genre, pays avec recherche en temps réel

Graphe interactif

react-force-graph-2d avec noeuds colorés par type

Animations fluides

Framer Motion pour transitions et interactions

Analyse des données

15+

Artistes

700+

Morceaux

600+

Albums

45+

Genres

Top artistes

Classement par nombre de morceaux dans la base

Top collaborations

Paires d'artistes avec le plus de morceaux en commun

Top genres

Genres les plus représentés par nombre d'artistes

Graphe des connexions

Visualisation interactive des liens entre artistes

Qualité des données

Pas de doublons

MBID (identifiants uniques MusicBrainz) + MERGE Cypher

Données manquantes

OPTIONAL MATCH + valeurs null gérées proprement

Rate limiting

1.1s entre chaque requête MusicBrainz

Normalisation

Types de noeuds et relations standardisés

Seed automatisé

Script d'import en masse avec gestion d'erreurs

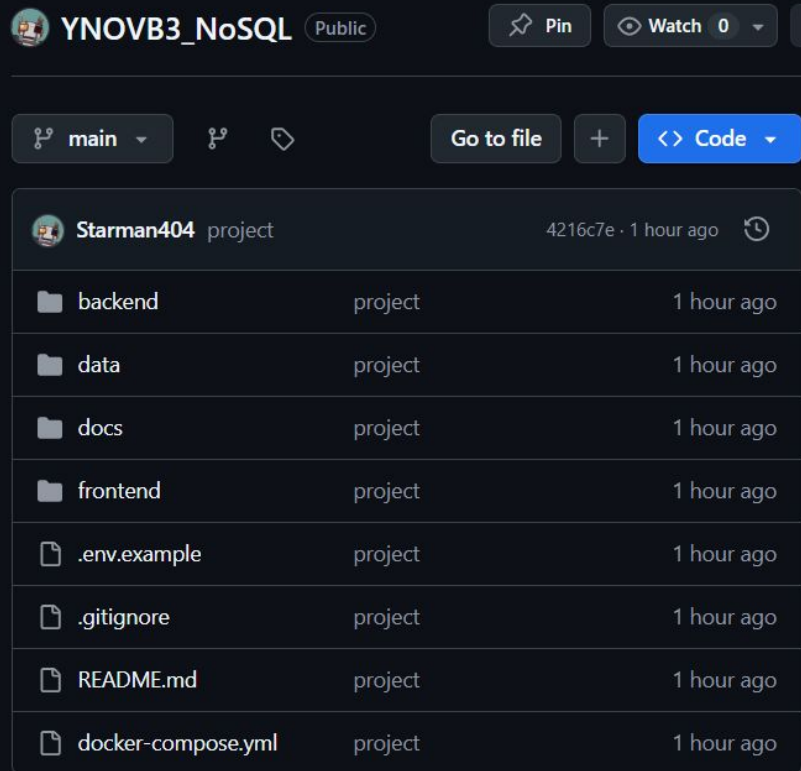
Conclusion

Objectifs atteints

- Application complète de A à Z
- Neo4j parfait pour les relations musicales
- MusicBrainz = source riche et gratuite
- Interface moderne et interactive

Limites

- Collaborations dépendantes de MusicBrainz
- Instance Aura Free limitée



Merci !

Des questions ?

Adlen Souci, Jade Hurpet, Sophie Aholou — B3 Dev & Data